

PZ-OV5640 摄像头模块开发手册

本手册将教大家如何在天马&麒麟开发板上使用 PZ-OV5640 摄像头模块。本章分为如下几部分内容：

- 1 PZ-OV5640 模块简介
- 2 硬件设计
- 3 软件设计
- 4 实验现象

1 PZ-OV5640 模块简介

1.1 特性参数

PZ-OV5640 是深圳普中科技推出的一款高性能 500W 像素高清摄像头模块。该模块采用 OmniVision 公司生产的一颗 1/4 英寸 CMOS QSXGA (2592*1944) 图像传感器：OV5640。PZ-OV5640 模块采用该 OV5640 传感器作为核心部件，集成有源晶振和 LD0，并且集成了自动对焦（AF）功能，具有非常高的性价比。

PZ-OV5640 模块的特点如下：

- 采用 $1.4\mu\text{m}\times 1.4\mu\text{m}$ 像素大小，并且使用 OmniBSI 技术以达到更高性能（高灵敏度、低串扰和低噪声）
- 自动图像控制功能：自动曝光（AEC）、自动白平衡（AWB）、自动消除灯光条纹、自动黑电平校准（ABLC）和自动带通滤波器（ABF）等。
- 支持图像质量控制：色饱和度调节、色调调节、gamma 校准、锐度和镜头校准等
- 标准的 SCCB 接口，兼容 IIC 接口
- 支持 RawRGB、RGB (RGB565/RGB555/RGB444)、CCIR656、YUV (422/420)、YCbCr (422) 和压缩图像（JPEG）输出格式
- 支持 QSXGA (500W) 图像尺寸输出，以及按比例缩小到其他任何尺寸
- 支持图像缩放、平移和窗口设置
- 支持图像压缩，即可输出 JPEG 图像数据
- 支持数字视频接口（DVP）和 MIPI 接口
- 支持自动对焦
- 自带嵌入式微处理器
- 集成有源晶振，无需外部提供时钟
- 集成 LD0，仅需提供 3.3V 电源即可正常工作

PZ-OV5640 模块各项参数如下图所示。

项目	说明
接口类型	数据接口：8 位数据 控制接口：SCCB (类 IIC 协议)
输出格式	RawRGB、RGB(RGB565/RGB555)、 GRB422、YUV(422/420)、YCbCr(422) 、JPEG 数据
输出位宽	8 位
输出像素	QXGA (2592*1944) 及以下 40*30的 任意尺寸
最大帧率	QXGA (2592*1944) : 15fps 1080P (1920*1080) : 30fps 720P (1280*720) : 60fps
传感器尺寸	1/4英寸
灵敏度	0.6V/Lux~sec
信噪比	36dB
动态范围	68dB
镜头光圈	F2.8
镜头视角	70°
镜头焦距	3.34mm
工作温度	-30℃~70℃
模块尺寸	24mm*32mm
自动对焦	支持单次自动对焦和持续自动对焦
输出窗口设置	支持输出窗口设置，可以匹配任意分辨率的液晶
缩放控制	支持缩放控制
电源电压	3.3V
I/O 口电平 ^①	2.8V LVTTTL，可兼容 3.3V
功耗	56mA

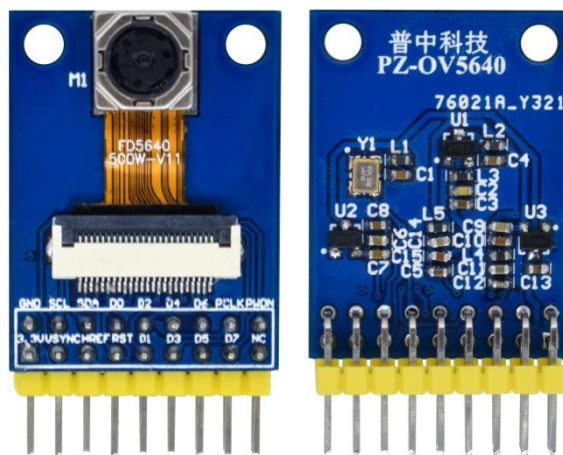
注①：模块 I/O 电压是 2.8V，不过对于 3.3V 系统，是可以直接兼容的。所以 3.3V 的 MCU 无需任何处理，直接连接模块即可。不过如果是 5V 的 MCU，建议在信号线上串接 1K 左右电阻，做限流处理。

1.2 使用说明

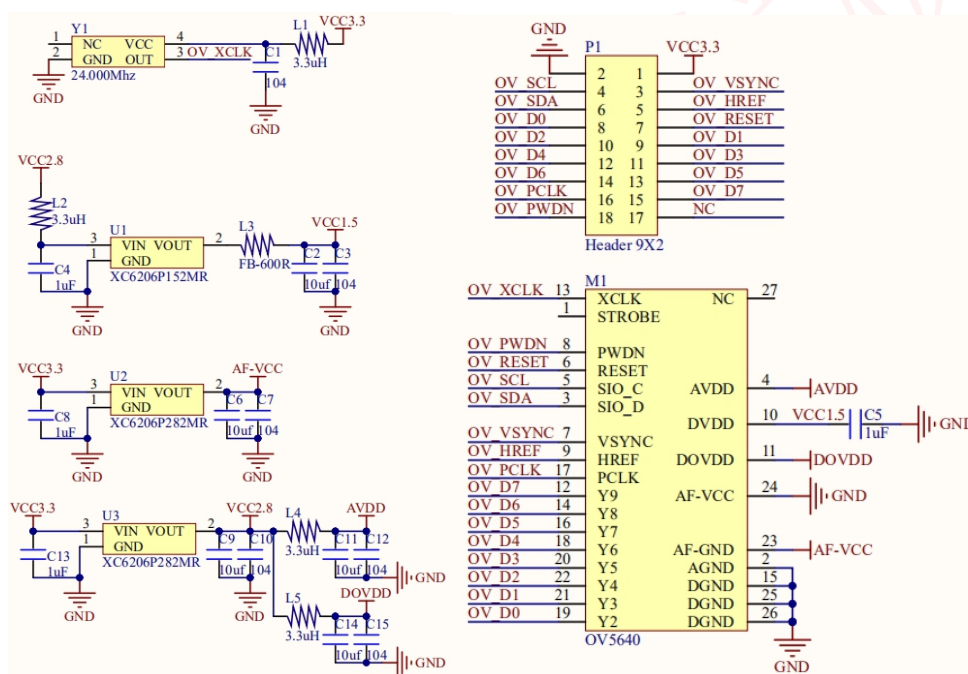
1.2.1 模块引脚说明

PZ-0V5640 摄像头模块通过 2*9 的弯排针 (2.54mm 间距) 同外部连接，模块可以与天马&麒麟等开发板直接对接，而普中 F407 最小系统开发板则可以通过杜邦线连接模块进行测试。所有普中 STM32F407 系列开发板都可使用该例程，用户可以直接在这些开发板上，对模块进行测试。

PZ-0V5640 摄像头模块外观如下图所示：



PZ-OV5640 摄像头模块原理图如下图所示：



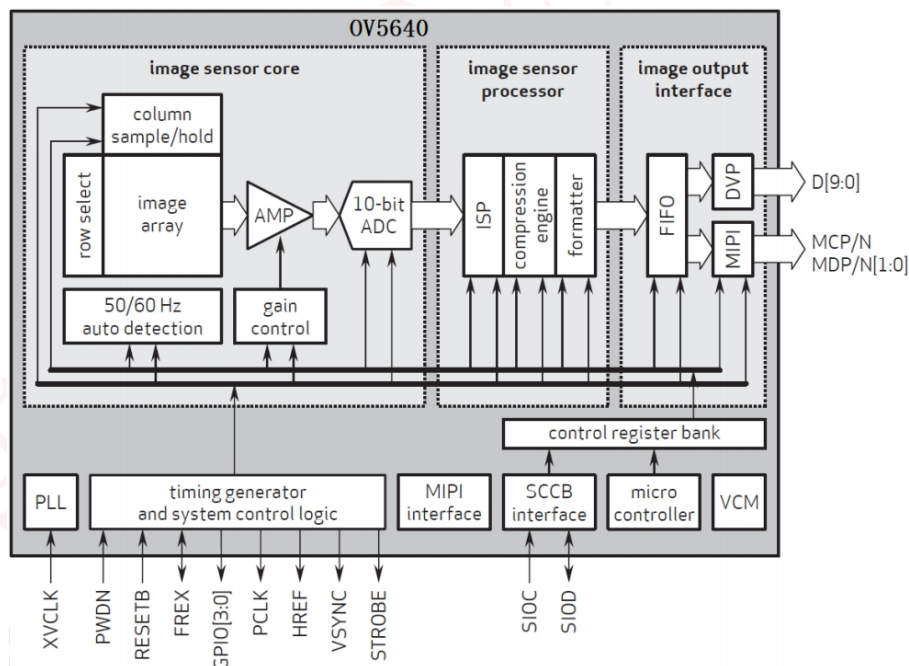
从图中可以看出，模块自带了 1.5V 和 2.8V 的稳压芯片，给 OV5640 供电，因此外部仅提供 3.3V 电压即可；同时自带了一个 24M 的有源晶振，所以模块不需要外部提供时钟。PZ-OV5640 摄像头模块通过一个 2*9 的排针（P1）同外部电路连接，各引脚的详细描述如下图所示：

序号	名称	说明
1	VCC	3.3V 电源输入脚
2	GND	地线
3	OV_VSYNC	帧同步信号 (OUT ^②)
4	OV_SCL	SCCB 时钟线 (IN ^①)
5	OV_HREF	行参考信号 (OUT)
6	OV_SDA	SCCB 数据线 (IN/OUT)
7	OV_RESET	复位信号 (低电平有效) (IN)
8-15	OV_D0~D7	数据线 (OUT)
16	OV_PCLK	像素时钟 (OUT)
17	NC	未用到
18	OV_PWDN	掉电模式使能 (高电平有效) (IN)

注①、②：IN，表示输入信号；OUT 表示输出信号；IN/OUT 表示即可以输出，也可以做输入信号。从上图可以看出，PZ-OV5640 摄像头模块，总共需要 15 个 IO 口驱动。

1.2.2 OV5640 内部功能结构介绍

OV5640 的功能框图如下图所示：



其中 image array 部分的尺寸，OV5640 的官方数据并没有给出具体的数字，其最大的有效输出尺寸为：2592*1944，即 500W 像素，我们根据官方提供的一些应用文档，发现其设置的 image array 最大为：2632*1951，所以，在接下来的介绍，我们设定其 image array 最大为 2632*1951。

1.2.3 串行摄像头控制总线(SCCB)简介

PZ-0V5640 摄像头模块的所有配置，都是通过 SCCB 总线来实现的，SCCB 全称是：Seril Camera Control Bus 即串行摄像头控制总线，它由两条数据线组成：一个是用于传输时钟信号的 SIO_C（即 0V_SCL），另一个是用于传输数据信号的 SIO_D（即 0V_SDA）。

SCCB 的传输协议与 IIC 协议极其相似，只不过 IIC 在每传输完一个字节后，接收数据的一方要发送一位的确认数据，而 SCCB 一次要传输 9 位数据，前 8 位为有用数据，而第 9 位数据在写周期中是 don' t care 位（即不必关心位），在读周期中是 NA 位。SCCB 定义数据传输的基本单元为相（phase），即一个相传输一个字节数据。

SCCB 只包括三种传输周期，即 3 相写传输周期（三个相依次为设备从地址，内存地址，所写数据），2 相写传输周期（两个相依次为设备从地址，内存地址）和 2 相读传输周期（两个相依次为设备从地址，所读数据）。当需要写操作时，应用 3 相写传输周期，当需要读操作时，依次应用 2 相写传输周期和 2 相读传输周期。

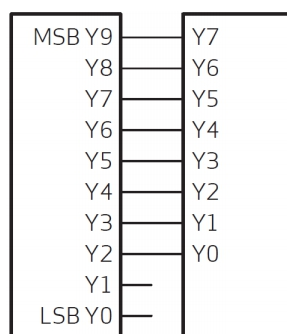
关于 SCCB 的详细介绍，请大家参考模块资料“\4--参考资料\《OmniVision Technologies Seril Camera Control Bus(SCCB) Specification.pdf》”这个文档。

OV5640 的初始化，需要配置大量寄存器，这里我们就不给大家多做介绍了，请大家参考模块资料“\4--参考资料\OV5640_CSP3_DS_2.01_Ruisipusheng.pdf”这个文档。

1.2.4 DVP 接口说明

PZ-0V5640 支持数字视频接口(DVP)和 MIPI 接口，因为我们的 STM32F4 使用的 DCMI 接口，仅支持 DVP 接口，所以，OV5640 必须使用 DVP 输出接口，才可以连接我们的 STM32F4 开发板。

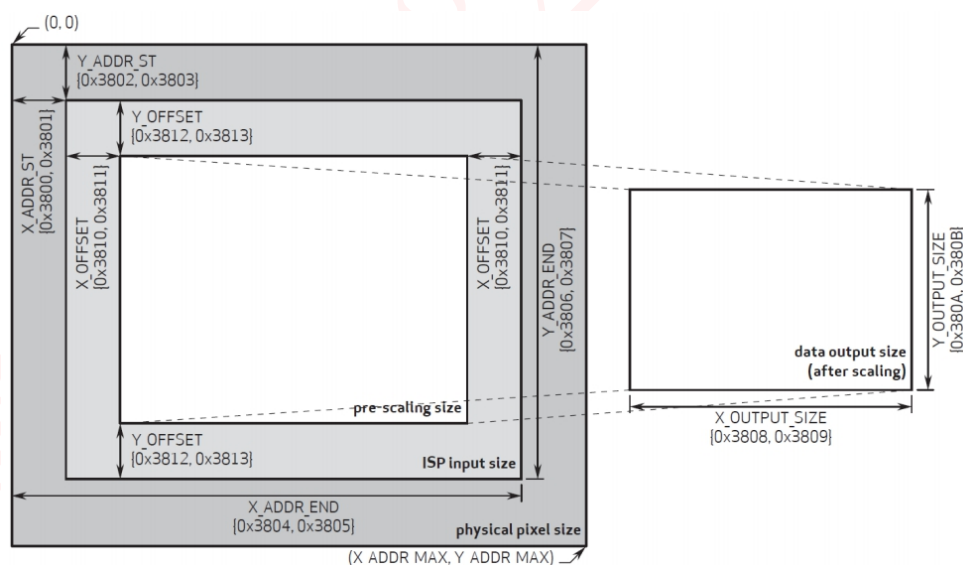
OV5640 提供一个 10 位 DVP 接口(支持 8 位接法)，其 MSB 和 LSB 可以程序设置先后顺序，PZ-0V5640 模块采用默认的 8 位连接方式，如下图所示：



OV5640 的寄存器通过 SCCB 时序访问并设置，SCCB 时序和 IIC 时序十分类似，在本章我们不做介绍，请大家参考资料《OmniVision Technologies Serial Camera Control Bus (SCCB) Specification》这个文档。

1.2.5 窗口设置说明

接下来，我们介绍一下 OV5640 的：ISP (Image Signal Processor) 输入窗口设置、预缩放窗口设置和输出大小窗口设置，这几个设置与我们的正常使用密切相关，有必要了解一下。他们的设置关系，如下图所示：



ISP 输入窗口设置 (ISP input size)

该设置允许用户设置整个传感器区域 (physical pixel size , 2632*1951) 的感兴趣部分，也就是在传感器里面开窗 (X_ADDR_ST、Y_ADDR_ST、X_ADDR_END 和 Y_ADDR_END)，开窗范围从 0*0~2632*1951 都可以设置，该窗口所设置的范围，将输入 ISP 进行处理。

ISP 输入窗口，通过：0X3800~0X3807 等 8 个寄存器进行设置，这些寄存

器的定义请看：OV5640_CSP3_DS_2.01_Ruisipusheng.pdf 这个文档（下同）。

预缩放窗口设置 (pre-scaling size)

该设置允许用户在 ISP 输入窗口的基础上，再次设置将要用于缩放的窗口大小。该设置仅在 ISP 输入窗口内进行 x/y 方向的偏移(X_OFFSET/Y_OFFSET)。通过：0X3810~0X3813 等 4 个寄存器进行设置。

输出大小窗口设置 (data output size)

该窗口是以预缩放窗口为原始大小，经过内部 DSP 进行缩放处理后，输出给外部的图像窗口大小。它控制最终的图像输出尺寸

(X_OUTPUT_SIZE/Y_OUTPUT_SIZE)。通过：0X3808~0X380B 等 4 个寄存器进行设置。注意：当输出大小窗口与预缩放窗口比例不一致时，图像将进行缩放处理（会变形），仅当两者比例一致时，输出比例才是 1:1（正常）。

上图中，右侧 data output size 区域，才是 OV5640 输出给外部的图像尺寸，也就是显示在 LCD 上面的图像大小。输出大小窗口与预缩放窗口比例不一致时，会进行缩放处理，在 LCD 上面看到的图像将会变形。

1.2.6 输出时序说明

接下来，我们介绍一下 OV5640 的图像数据输出时序。首先我们简单介绍一些定义：

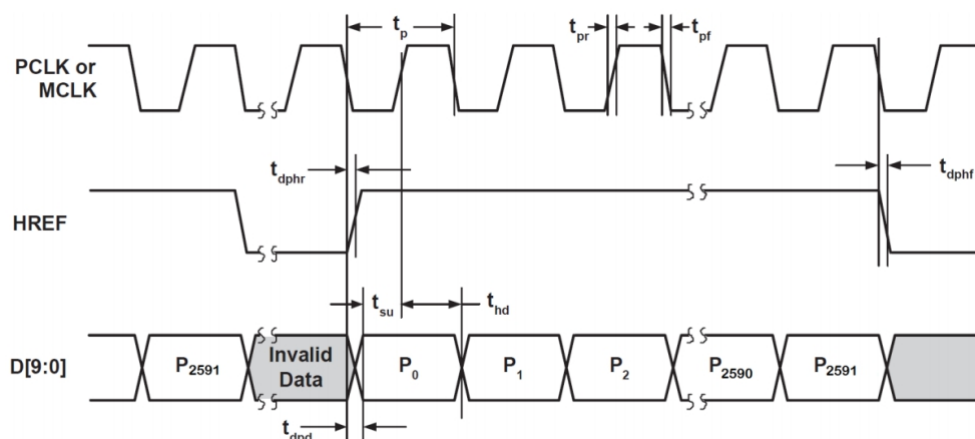
QSXGA，这里指：分辨率为 2592*1944 的输出格式，类似的还有：QXGA(2048*1536)、UXGA(1600*1200)、SXGA(1280*1024)、WXGA+(1440*900)、WXGA(1280*800)、XGA(1024*768)、SVGA(800*600)、VGA(640*480)、QVGA(320*240)和 QQVGA(160*120)等。

PCLK，即像素时钟，一个 PCLK 时钟，输出一个像素(或半个像素)。

VSYNC，即帧同步信号。

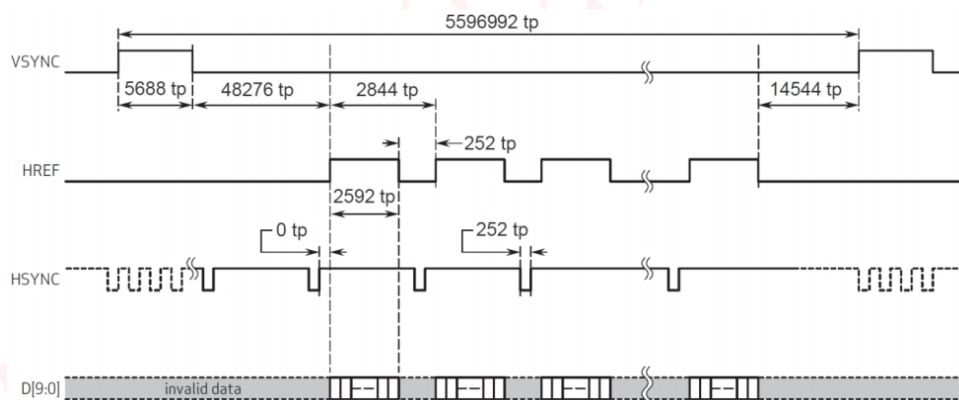
HREF /HSYNC，即行同步信号。

OV5640 的图像数据输出(通过 Y[9:0])就是在 PCLK,VSYNC 和 HREF/HSYNC 的控制下进行的。首先看看行输出时序，如下图所示：



从上图可以看出，图像数据在 HREF 为高的时候输出，当 HREF 变高后，每一个 PCLK 时钟，输出一个 8 位/10 位数据。我们采用 8 位接口，所以每个 PCLK 输出 1 个字节，且在 RGB/YUV 输出格式下，每个 $t_p=2$ 个 T_{pclk} ，如果是 Raw 格式，则一个 $t_p=1$ 个 T_{pclk} 。比如我们采用 QSXGA 时序，RGB565 格式输出，每 2 个字节组成一个像素的颜色（低字节在前，高字节在后），这样每行输出总共有 $2592*2$ 个 PCLK 周期，输出 $2592*2$ 个字节。

再来看看帧时序（QSXGA 模式），如下图所示：



上图清楚的表示了 OV5640 在 QSXGA 模式下的数据输出。我们按照这个时序去读取 OV5640 的数据，就可以得到图像数据。

1.2.7 自动对焦（Auto Focus）说明

OV5640 由内置微型控制器完成自动对焦，并且 VCM（Voice Coil Motor，即音圈马达）驱动器也已集成在传感器内部。微型控制器的控制固件（firmware）从主机下载。当固件运行后，内置微型控制器从 OV5640 传感器读得自动对焦所需的信息，计算并驱动 VCM 马达带动镜头到达正确的对焦位置。主机可以通过

IIC 命令控制微型控制器的各种功能。

OV5640 的自动对焦命令（通过 SCCB 总线发送），如下图所示：

地址	寄存器名	描述	值
0X3022	CMD_MAIN	AF 主命令寄存器	0X03: 触发单次自动对焦过程 0X04: 启动持续自动对焦过程 0X06: 暂停自动对焦过程 0X08: 释放马达回到初始状态 0X12: 设置对焦区域 0X00: 命令完成
0X3023	CMD_ACK	命令确认	0X00: 命令完成 0X01: 命令执行中
0X3029	FW_STATUS	对焦状态	0X7F: 固件下载完成，但未执行，可能是：固件有问题/微控制器关闭 0X7E: 固件初始化中 0X70: 释放马达，回到初始状态 0X00: 正在自动对焦 0X10: 自动对焦完成

OV5640 内部的微控制器收到自动对焦命令后会自动将 CMD_MAIN（0X3022）寄存器数据清零，当命令完成后会将 CMD_ACK（0X3023）寄存器数据清零。

自动对焦（AF）过程

1、在第一次进入图像预览的时候（图像可以正常输出时），下载固件（firmware）

2、拍照前，自动对焦，对焦完成后，拍照

3、拍照完毕，释放马达到初始状态

接下来，我们分别说明。

①下载固件

OV5640 初始化完成后，就可以下载 AF 自动对焦固件了，其操作和下载初始化参数类似，AF 固件下载地址为：0X8000，初始化数组由厂家提供（本例程该数组保存在 ov5640af.h 里面），下载固件完成后，通过检查 0X3029 寄存器的值，来判断固件状态（等于 0X70，说明正常）。

②自动对焦

OV5640 支持单次自动对焦和持续自动对焦，通过 0X3022 寄存器控制。单次自动对焦过程如下：

- 1，将 0X3022 寄存器写为 0X03，开始单点对焦过程。
- 2，读取寄存器 0X3029，如果返回值为 0X10，代表对焦已完成。
- 3，写寄存器 0X3022 为 0X06，暂停对焦过程，使镜头将保持在此对焦位置。

其中，前两步是必须的，第三步，可以不要，因为单次自动对焦完成以后，就不会继续自动对焦了，镜头也就不会动了。

持续自动对焦过程如下：

- 1，将 0X22 寄存器写为 0X08，释放马达到初始位置（对焦无穷远）
- 2，将 0X3022 寄存器写为 0X04，启动持续自动对焦过程。
- 3，读取寄存器 0X3023，等待命令完成。
- 4，当 0V5640 每次检测到失焦时，就会自动进行对焦（一直检测）。

③释放马达，结束自动对焦

最后，在拍照完成，或者需要结束自动对焦的时候，我们对在寄存器 0X3022 写入 0X08，即可释放马达，结束自动对焦。

最后说一下 0V5640 的图像数据格式，我们一般用 2 种输出方式：RGB565 和 JPEG。当输出 RGB565 格式数据的时候，时序完全就是上面两幅图介绍的关系。以满足不同需要。而当输出数据是 JPEG 数据的时候，同样也是这种方式输出（所以数据读取方法一模一样），不过 PCLK 数目大大减少了，且不连续，输出的数据是压缩后的 JPEG 数据，输出的 JPEG 数据以：0XFF,0XD8 开头，以 0XFF,0XD9 结尾，且在 0XFF,0XD8 之前，或者 0XFF,0XD9 之后，会有不定数量的其他数据存在（一般是 0），这些数据我们直接忽略即可，将得到的 0XFF,0XD8~0XFF,0XD9 之间的数据，保存为.jpg/.jpeg 文件，就可以直接在电脑上打开看到图像了。

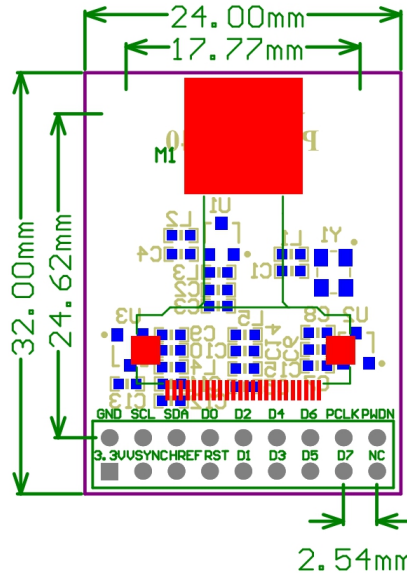
0V5640 自带的 JPEG 输出功能，大大减少了图像的数据量，使得其在网络摄像头、无线视频传输等方面具有很大的优势。0V5640 我们就介绍到这，关于 0V5640 更详细的介绍，请大家参考资料“\4--参考资料\0V5640_CSP3_DS_2.01_Ruisipusheng.pdf”。

1.2.8 模块使用注意事项

PZ-0V5640 摄像头模块使用的是一体化的 0V5640 模组，本身就含有调焦镜头。当显示画面模糊时，可通过旋转模组镜头来调节清晰度。如果用手调节比较困难时，可以用镊子夹住镜头旋转。默认已经是调节好的。

1.3 结构尺寸

PZ-0V5640 摄像头模块尺寸结构如下图所示：



1.4 STM32F4 DCMI 接口简介

STM32F4 自带了一个数字摄像头 (DCMI) 接口，该接口是一个同步并行接口，能够接收外部 8 位、10 位、12 位或 14 位 CMOS 摄像头模块发出的高速数据流。可支持不同的数据格式：YCbCr4:2:2/RGB565 逐行视频和压缩数据 (JPEG)。

STM32F4 DCMI 接口特点：

- 8 位、10 位、12 位或 14 位并行接口
- 内嵌码/外部行同步和帧同步
- 连续模式或快照模式
- 裁剪功能
- 支持以下数据格式：
 - 1, 8/10/12/14 位逐行视频：单色或原始拜尔 (Bayer) 格式
 - 2, YCbCr 4:2:2 逐行视频
 - 3, RGB 565 逐行视频
 - 4, 压缩数据：JPEG

DCMI 接口包括如下一些信号：

- 1, 数据输入 (D[0:13])，用于接摄像头的数据输出，接 OV2640 我们只用

了 8 位数据。

2, 水平同步（行同步）输入（HSYNC），用于接摄像头的 HSYNC/HREF 信号。

3, 垂直同步（场同步）输入（VSYNC），用于接摄像头的 VSYNC 信号。

4, 像素时钟输入（PIXCLK），用于接摄像头的 PCLK 信号。

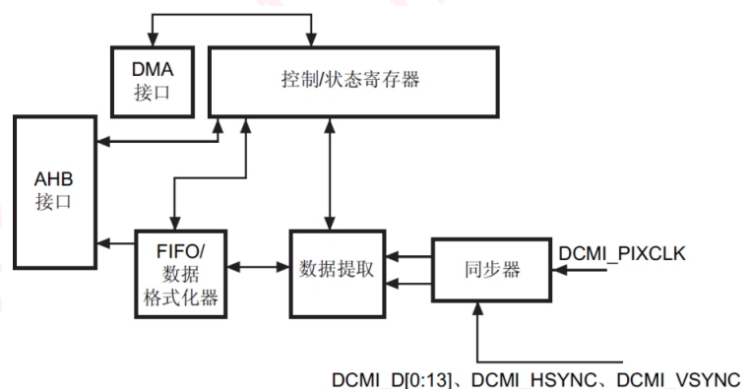
DCMI 接口是一个同步并行接口，可接收高速（可达 54 MB/s）数据流。该接口包含多达 14 条数据线(D13-D0)和一条像素时钟线(PIXCLK)。像素时钟的极性可以编程，因此可以在像素时钟的上升沿或下降沿捕获数据。

DCMI 接收到的摄像头数据被放到一个 32 位数据寄存器(DCMI_DR)中，然后通过通用 DMA 进行传输。图像缓冲区由 DMA 管理，而不是由摄像头接口管理。

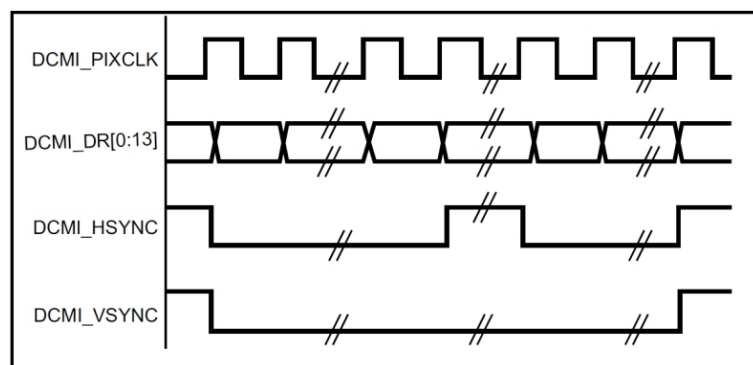
从摄像头接收的数据可以按行/帧来组织（原始 YUV/RGB/拜尔模式），也可以是一系列 JPEG 图像。要使能 JPEG 图像接收，必须将 JPEG 位（DCMI_CR 寄存器的位 3）置 1。

数据流可由可选的 HSYNC（水平同步）信号和 VSYNC（垂直同步）信号硬件同步，或者通过数据流中嵌入的同步码同步。

STM32F4 DCMI 接口的框图如下图所示：



DCMI 接口的数据与 PIXCLK（即 PCLK）保持同步，并根据像素时钟的极性在像素时钟上升沿/下降沿发生变化。HSYNC（HREF）信号指示行的开始/结束，VSYNC 信号指示帧的开始/结束。DCMI 信号波形如下图所示：



上图中，对应设置为：DCMI_PIXCLK 的捕获沿为下降沿，DCMI_HSYNC 和 DCMI_VSYNC 的有效状态为 1，注意，这里的有效状态实际上对应的是指示数据在并行接口上无效时，HSYNC/VSYNC 引脚上面的引脚电平。

本实验我们用到 DCMI 的 8 位数据宽度，通过设置 DCMI_CR 中的 EDM[1:0]=00 设置。此时 DCMI_D0~D7 有效，DCMI_D8~D13 上的数据则忽略，这个时候，每次需要 4 个像素时钟来捕获一个 32 位数据。捕获的第一个数据存放在 32 位字的 LSB 位置，第四个数据存放在 32 位字的 MSB 位置，捕获数据字节在 32 位字中的排布如下图所示：

字节地址	31:24	23:16	15:8	7:0
0	D _{n+3} [7:0]	D _{n+2} [7:0]	D _{n+1} [7:0]	D _n [7:0]
4	D _{n+7} [7:0]	D _{n+6} [7:0]	D _{n+5} [7:0]	D _{n+4} [7:0]

从图中可以看出，STM32F4 的 DCMI 接口，接收的数据是低字节在前，高字节在后的，所以，要求摄像头输出数据也是低字节在前，高字节在后才可以，否则就还得程序上处理字节顺序，会比较麻烦。

DCMI 接口支持 DMA 传输，当 DCMI_CR 寄存器中的 CAPTURE 位置 1 时，激活 DMA 接口。摄像头接口每次在其寄存器中收到一个完整的 32 位数据块时，都将触发一个 DMA 请求。

DCMI 接口支持两种同步方式：内嵌码同步和硬件（HSYNC 和 VSYNC）同步。我们简单介绍下硬件同步，详细介绍可以参考开发板资料中《STM32F4xx 中文数据手册》第 13.5.3 节。

硬件同步模式下将使用两个同步信号（HSYNC/VSYNC）。根据摄像头模块/模式的不同，可能在水平/垂直同步期间内发送数据。由于系统会忽略 HSYNC/VSYNC 信号有效电平期间内接收的所有数据，HSYNC/VSYNC 信号相当于消隐信号。

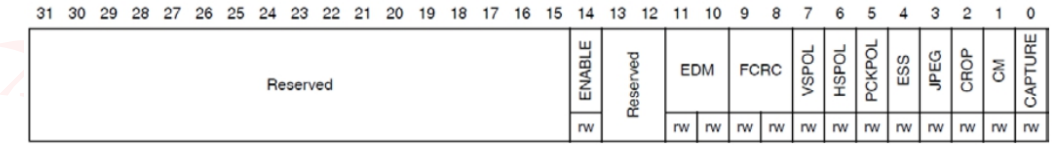
为了正确地将图像传输到 DMA/RAM 缓冲区，数据传输将与 VSYNC 信号同

步。选择硬件同步模式并启用捕获（DCMI_CR 中的 CAPTURE 位置 1）时，数据传输将与 VSYNC 信号的无效电平同步（开始下一帧时）。之后传输便可以连续执行，由 DMA 将连续帧传输到多个连续的缓冲区或一个具有循环特性的缓冲区。为了允许 DMA 管理连续帧，每一帧结束时都将激活 VSIF（垂直同步中断标志，即帧中断），我们可以利用这个帧中断来判断是否有一帧数据采集完成，方便处理数据。

DCMI 接口的捕获模式支持：快照模式和连续采集模式。一般我们使用连续采集模式，通过 DCMI_CR 中的 CM 位设置。另外，DCMI 接口还支持实现了 4 个字深度的 FIFO，配有一个简单的 FIFO 控制器，每次摄像头接口从 AHB 读取数据时读指针递增，每次摄像头接口向 FIFO 写入数据时写指针递增。因为没有溢出保护，如果数据传输率超过 AHB 接口能够承受的速率，FIFO 中的数据就会被覆盖。如果同步信号出错，或者 FIFO 发生溢出，FIFO 将复位，DCMI 接口将等待新的数据帧开始。

关于 DCMI 接口的其他特性，我们这里就不再介绍了，请大家参考开发板资料《STM32F4xx 中文参考手册》第 13 章相关内容。

本实验我们将 OV2640 默认配置为 UXGA 输出，也就是 1600*1200 的分辨率，输出信号设置为：VSYNC 高电平有效，HREF 高电平有效，输出数据在 PCLK 的下降沿输出（即上升沿的时候，MCU 才可以采集）。这样，STM32F4 的 DCMI 接口就必须设置为：VSYNC 低电平有效、HSYNC 低电平有效和 PIXCLK 上升沿有效，这些设置都是通过 DCMI_CR 寄存器控制的，该寄存器描述如下图所示：



ENABLE: 该位用于设置是否使能 DCMI，不过，在使能之前，必须将其他配置设置好。

FCRC[1:0]: 这两个位用于帧率控制，我们捕获所有帧，所以设置为 00 即可。

VSPOL: 该位用于设置垂直同步极性，也就是 VSYNC 引脚上面，数据无效时的电平状态，根据前面说所，我们应该设置为 0。

HSPOL: 该位用于设置水平同步极性，也就是 HSYNC 引脚上面，数据无效时

的电平状态，同样应该设置为 0。

PCKPOL: 该位用于设置像素时钟极性，我们用上升沿捕获，所以设置为 1。

CM: 该位用于设置捕获模式，我们用连续采集模式，所以设置为 0 即可。

CAPTURE: 该位用于使能捕获，我们设置为 1。该位使能后，将激活 DMA，DCMI 等待第一帧开始，然后生成 DMA 请求将收到的数据传输到目标存储器中。

注意：该位必须在 DCMI 的其他配置（包括 DMA）都设置好了之后，才设置！！

DCMI_CR 寄存器的其他位，我们就不介绍了，另外 DCMI 的其他寄存器这里也不再介绍，请大家参考开发板中资料《STM32F4xx 中文参考手册》第 13.8 节。

1.5 STM32F4 DCMI 配置步骤

最后，我们来看下用 DCMI 驱动 OV5640 的步骤：

（1）配置 OV5640 控制引脚，并配置 OV5640 工作模式。

在启动 DCMI 之前，我们先设置好 OV5640。OV5640 通过 OV_SCL 和 OV_SDA 进行寄存器配置，同时还有 OV_PWDN/OV_RESET 等信号，我们也需要配置对应 IO 状态，先设置 OV_PWDN=0，退出掉电模式，然后拉低 OV_RESET 复位 OV5640，之后再设置 OV_RESET 为 1，结束复位，然后就是对 OV5640 的大把寄存器进行配置了。然后，可以根据我们的需要，设置成 RGB565 输出模式，还是 JPEG 输出模式。

（2）配置相关引脚的模式和复用功能（AF13），使能时钟。

OV5640 配置好之后，再设置 DCMI 接口与摄像头模块连接的 IO 口，使能 IO 和 DCMI 时钟，然后设置相关 IO 口为复用功能模式，复用功能选择 AF13(DCMI 复用)。

（3）配置 DCMI 相关设置。

这一步，主要通过 DCMI_CR 寄存器设置，包括 VSPOL/HSPOL/PCKPOL/数据宽度等重要参数，都在这一步设置，同时我们也开启帧中断，编写 DCMI 中断服务函数，方便进行数据处理（尤其是 JPEG 模式的时候）。不过对于 CAPTURE 位，我们等待 DMA 配置好之后再设置，另外对于 OV5640 输出的 JPEG 数据，我们也不使用 DCMI 的 JPEG 数据模式（实测设置不设置都一样），而是采用正常模式，直接采集。

(4) 配置 DMA。

本实验采用连续模式采集，并将采集到的数据输出到 LCD（RGB565 模式）或内存（JPEG 模式），所以源地址都是 DCMI_DR，而目的地址可能是 TFTLCD->DATA 或者 SRAM 的地址。DCMI 的 DMA 传输采用的是 DMA2 数据流 1 的通道 1 来实现的，关于 DMA 的介绍，请大家参考我们开发板开发攻略教程。

(5) 设置 OV5640 的图像输出大小，使能 DCMI 捕获。

图像输出大小设置，分两种情况：在 RGB565 模式下，我们根据 LCD 的尺寸，设置输出图像大小，以实现全屏显示（图像可能因缩放而变形）；在 JPEG 模式下，我们可以自由设置输出图像大小（可不缩放）；最后，开启 DCMI 捕获，即可正常工作了。

2 硬件设计

本章实验功能简介：开机后，初始化摄像头模块（OV5640），如果初始化成功，则提示选择模式：RGB565 模式，或者 JPEG 模式。KEY0 用于选择 RGB565 模式，KEY1 用于选择 JPEG 模式。

当使用 RGB565 时，输出图像（固定为：WXGA）将经过缩放处理（完全由 OV5640 的 DSP 控制），显示在 LCD 上面（默认开启连续自动对焦）。我们可以通过 KEY_UP 按键选择：1:1 显示，即不缩放，图片不变形，但是显示区域小（液晶分辨率大小），或者缩放显示，即将 1280*800 的图像压缩到液晶分辨率尺寸显示，图片变形，但是显示了整个图片内容。通过 KEY0 按键，可以设置对比度；KEY1 按键，可以启动单次自动对焦；KEY2 按键，可以设置特效。

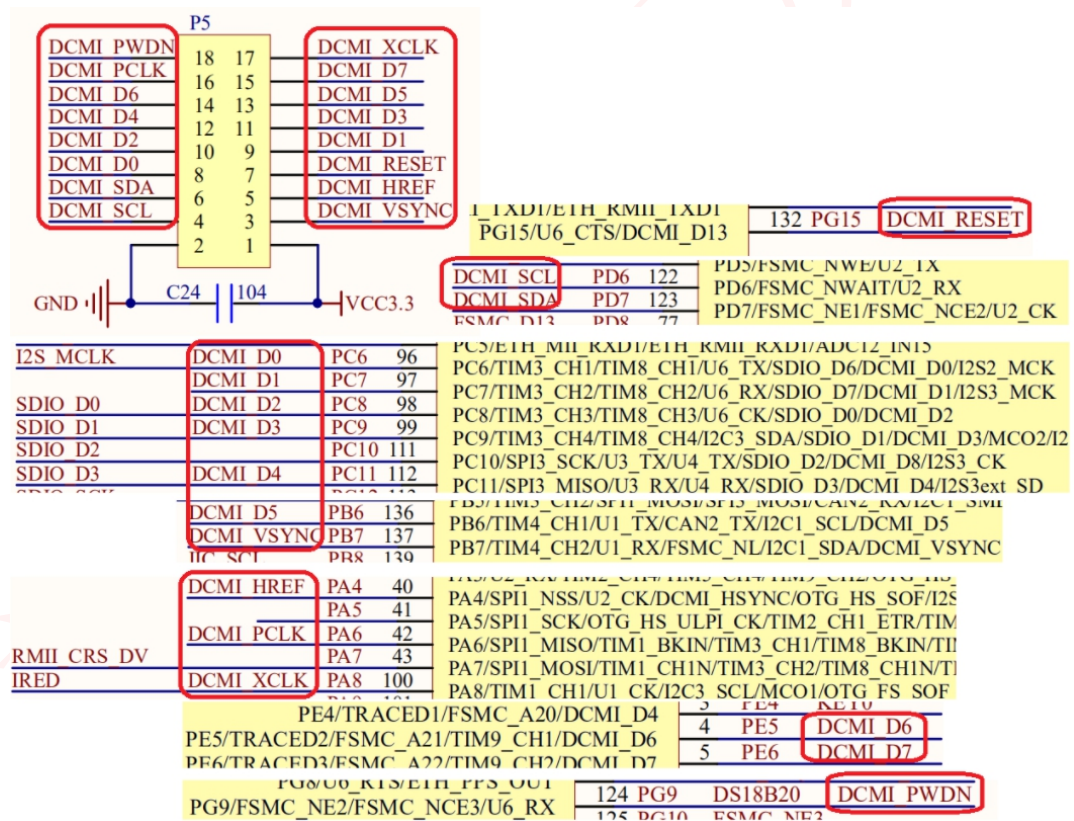
当使用 JPEG 模式时，图像可以设置尺寸（QQVGA~VGA），采集到的 JPEG 数据将先存放到 STM32F4 的 RAM 内存里面，每当采集到一帧数据，就会关闭 DMA 传输，然后将采集到的数据发送到串口 1（此时可以通过上位机软件（CAM.exe）接收，并显示图片），之后再重新启动 DMA 传输。我们可以通过 KEY_UP 设置输出图片的尺寸（QQVGA~VGA）。通过 KEY0 按键，可以设置对比度；KEY1 按键，可以启动单次自动对焦；KEY2 按键，可以设置特效。

DS0 指示程序运行状态，DS1 用于指示帧中断。

本实验使用到硬件资源如下：

- (1) DS0、DS1 指示灯
- (2) KEY_UP、KEY0、KEY1 和 KEY2 按键
- (3) 串口 1
- (4) TFTLCD 模块
- (5) PZ-0V5640 摄像头模块

这些资源，除了最后一个，其他的我们在开发板中开发攻略教程都介绍过。这里我们重点介绍下天马&麒麟开发板的摄像头接口与 PZ-0V5640 摄像头模块的连接。在开发板的左上角的 2*9 的 P5 排座，是摄像头模块/OLED 模块共用接口。本实验，我们只需要将 PZ-0V5640 摄像头模块插入这个接口即可，该接口与 STM32 的连接关系如下图所示：



从上图可以看出，OV2640 摄像头模块的各信号脚与 STM32 的连接关系为：

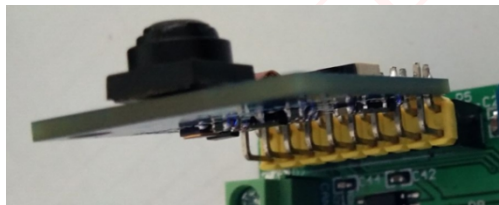
DCMI_VSYNC 接 PB7；
DCMI_HREF 接 PA4；
DCMI_PCLK 接 PA6；
DCMI_SCL 接 PD6；

DCMI_SDA 接 PD7;
DCMI_RESET 接 PG15;
DCMI_PWDN 接 PG9;
DCMI_XCLK 接 PA8 (本章未用到);
DCMI_D[7:0]接 PE6/PE5/PB6/PC11/PC9/PC8/PC7/PC6;

这些线的连接，天马&麒麟开发板的内部已经连接好了，我们只需要将 PZ-0V5640 摄像头模块插上去就好了。板载接口如下：



特别注意：模块插入开发板摄像头接口时注意方向，可对照板载接口背面 PCB 丝印电源脚位即可，PZ-0V5640 插入开发板图如下：



注意：DCMI 摄像头接口和 I2S 接口、DAC、SDIO 以及 DS18B20 等有冲突，使用的时候，必须分时复用才可以，不可同时使用。

3 软件设计

打开本实验工程目录，我们在 APP 文件夹下新建了 OV5640 和 DCMI 两个文件夹。在 ov5640 文件夹下新建了 ov5640.c、sccb.c、ov5640.h、sccb.h、ov5640cfg.h 等 5 个文件，并同时在工程中将这个文件夹加入头文件包含路径。在 dcmi 文件夹新建了 dcmi.c 和 dcmi.h 两个文件，也将这个文件夹加入头文件包含路径。

DCMI 接口相关的库函数分布在 stm32f4xx_dcmi.c 文件以及对应的头文件 stm32f4xx_dcmi.h 中。我们工程引入了这两个文件。

本实验总共新增了 7 个文件，代码比较多，我们就不一一列出了，仅挑几个重要的地方进行讲解。

3.1 ov5640.c

首先，我们来看 ov5640.c 里面的 OV5640_Init 函数，该函数代码如下：

```
1.  //初始化 OV5640
2.
3.  //返回值:0,成功
4.  //    其他,错误代码
5.  u8 OV5640_Init(void)
6.  {
7.      u16 i=0;
8.      u16 reg;
9.      //设置 IO
10.     GPIO_InitTypeDef  GPIO_InitStructure;
11.
12.     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOG, ENABLE);
13.     //GPIOG9,15 初始化设置
14.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9|GPIO_Pin_15;//PG9,15 推挽输出
15.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT; //推挽输出
16.     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;//推挽输出
17.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;//100MHz
18.     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;//上拉
19.     GPIO_Init(GPIOG, &GPIO_InitStructure);//初始化
20.
21.     OV5640_RST=0;           //必须先拉低 OV5640 的 RST 脚,再上电
22.     delay_ms(20);
23.     OV5640_PWDN=0;         //POWER ON
24.     delay_ms(5);
25.     OV5640_RST=1;          //结束复位
26.     delay_ms(20);
27.     SCCB_Init();            //初始化 SCCB 的 IO 口
28.     delay_ms(5);
29.     reg=OV5640_RD_Reg(OV5640_CHIPIDH); //读取 ID 高八位
30.     reg<<=8;
31.     reg|=OV5640_RD_Reg(OV5640_CHIPIDL); //读取 ID 低八位
32.     if(reg!=OV5640_ID)
33.     {
34.         printf("ID:%d\r\n",reg);
35.         return 1;
36.     }
37.     OV5640_WR_Reg(0x3103,0X11); //system clock from pad, bit[1]
38.     OV5640_WR_Reg(0X3008,0X82); //软复位
39.     delay_ms(10);
40.     //初始化 OV5640
```

```

41.     for(i=0;i<sizeof(ov5640_init_reg_tbl)/4;i++)
42.     {
43.         OV5640_WR_Reg(ov5640_init_reg_tbl[i][0],ov5640_init_reg_tbl[i][1]);
44.     }
45.     //检查闪光灯是否正常
46.     OV5640_Flash_Ctrl(1);//打开闪光灯
47.     delay_ms(50);
48.     OV5640_Flash_Ctrl(0);//关闭闪光灯
49.     return 0x00;    //ok
50. }

```

此部分代码先初始化 OV5640 相关的 IO 口（包括 SCCB_Init），然后最主要的是完成 OV5640 的寄存器序列初始化。OV5640 的寄存器很多（一百几十个），配置比较麻烦，幸好厂家有提供参考配置序列

《OV5640_camera_module_software_application_notes_1.3_Sonix.pdf, 》），本章我们用到的配置序列，存放在 ov5640_init_reg_tbl 这个数组里面，该数组是一个 2 维数组，存储初始化序列寄存器及其对应的值，该数组存放在 ov5640cfg.h 里面。

另外，在 ov5640.c 里面，还有几个函数比较重要，这里不贴代码了，只介绍功能：

OV5640_ImageWin_Set 函数，该函数用于设置 ISP 输入窗口；

OV5640_OutSize_Set 函数，用于设置预缩放窗口和输出大小窗口；

OV5640_Focus_Init 函数，用于初始化自动对焦功能；

OV5640_Focus_Single 函数，用于实现一次自动对焦；

OV5640_Focus_Constant 函数，用于开启持续自动对焦功能；

OV5640_ImageWin_Set 和 OV5640_OutSize_Set 这就是我们在 1.2 节所介绍的 3 个窗口的设置，他们共同决定了图像的输出。

3.2 ov5640cfg.h

接下来，我们看看 ov2640cfg.h 里面 ov2640_uxga_init_reg_tbl 的内容，ov2640cfg.h 文件的代码如下：

```

1.     //JPEG 配置.7.5 帧
2.     //最大支持 2592*1944 的 JPEG 图像输出
3.     const u16 OV5640_jpeg_reg_tbl[][2]=
4.     {

```



```

5.      0x4300, 0x30, // YUV 422, YUYV
6.      .....//省略部分代码
7.      0x3503, 0x00, // AEC/AGC on
8.  };
9.      //RGB565 配置.15 帧
10.     //最大支持 1280*800 的 RGB565 图像输出
11.     const u16 ov5640_rgb565_reg_tbl[][2]=
12.     {
13.         0x4300, 0x6F,
14.         .....//省略部分代码
15.         0x3503, 0x00, // AEC/AGC on
16.     };
17.     //OV5640 初始化寄存器序列表
18.     const u16 ov5640_init_reg_tbl[][2]=
19.     {
20.         // 24MHz input clock, 24MHz PCLK
21.         0x3008, 0x42, // software power down, bit[6]
22.         .....//省略部分代码
23.         0x4740, 0x21, //VSYNC 高有效
24.     };

```

以上代码，我们省略了很多（全部贴出来太长了），里面总共有 3 个数组。我们大概了解下数组结构，每个数组条目的第一个字节为寄存器号（也就是寄存器地址），第二个字节为要设置的值，比如{0x4300, 0x30}，就表示在 0x4300 地址，写入 0x30 这个值。

这里面：ov5640_init_reg_tbl 数组，用于初始化 OV5640，该数组必须最先进行配置；ov5640_rgb565_reg_tbl 数组，用于设置 OV5640 的输出格式为 RGB565，分辨率为 1280*800，帧率为 15 帧，在 RGB 模式下使用；OV5640_jpeg_reg_tbl 用于设置 OV5640 的输出格式为 JPEG，分辨率为 2592*1944，帧率为 7.5 帧，在 JPEG 模式下使用。

3.3 dcmi.c

接下来，我们看看 dcmi.c 里面的代码，如下：

```

1.     #include "dcmi.h"
2.     #include "tftlcd.h"
3.     #include "led.h"
4.     #include "ov5640.h"
5.
6.

```



```

7.   u8 ov_frame=0;                                //帧率
8.   extern void jpeg_data_process(void);           //JPEG 数据处理函数
9.
10.  DCMI_InitTypeDef DCMI_InitStructure;
11.
12.  //DCMI 中断服务函数
13.  void DCMI_IRQHandler(void)
14.  {
15.      if(DCMI_GetITStatus(DCMI_IT_FRAME)==SET)//捕获到一帧图像
16.      {
17.          jpeg_data_process();    //jpeg 数据处理
18.          DCMI_ClearITPendingBit(DCMI_IT_FRAME);//清除帧中断
19.          LED2=!LED2;
20.          ov_frame++;
21.      }
22.  }
23.  //DCMI DMA 配置
24.  //DMA_Memory0BaseAddr:存储器地址    将要存储摄像头数据的内存地址(也可以是外设地址)
25.  //DMA_BufferSize:存储器长度    0~65535
26.  //DMA_MemoryDataSize:存储器位宽
27.  //DMA_MemoryDataSize:      存          储          器          位
    宽      @defgroup DMA_memory_data_size :DMA_MemoryDataSize_Byte/DMA_MemoryDataSize_HalfWord/DMA_
    MemoryDataSize_Word
28.  //DMA_MemoryInc:      存          储          器          增          长          方
    式      @defgroup DMA_memory_incremented_mode /** @defgroup DMA_memory_incremented_mode : DMA_Mem
    oryInc_Enable/DMA_MemoryInc_Disable
29.  void DCMI_DMA_Init(u32 DMA_Memory0BaseAddr,u16 DMA_BufferSize,u32 DMA_MemoryDataSize,u32 DM
    A_MemoryInc)
30.  {
31.      DMA_InitTypeDef DMA_InitStructure;
32.
33.      RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_DMA2,ENABLE);//DMA2 时钟使能
34.      DMA_DeInit(DMA2_Stream1);
35.      while (DMA_GetCmdStatus(DMA2_Stream1) != DISABLE){}//等待 DMA2_Stream1 可配置
36.
37.      /* 配置 DMA Stream */
38.      DMA_InitStructure.DMA_Channel = DMA_Channel_1; //通道 1 DCMI 通道
39.      DMA_InitStructure.DMA_PeripheralBaseAddr = (u32)&DCMI->DR;//外设地址为:DCMI->DR
40.      DMA_InitStructure.DMA_Memory0BaseAddr = DMA_Memory0BaseAddr;//DMA 存储器 0 地址
41.      DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralToMemory;//外设到存储器模式
42.      DMA_InitStructure.DMA_BufferSize = DMA_BufferSize;//数据传输量
43.      DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;//外设非增量模式
44.      DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc;//存储器增量模式

```

```

45.     DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Word;//外设数据长度:32
    位
46.     DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize;//存储器数据长度
47.     DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;// 使用循环模式
48.     DMA_InitStructure.DMA_Priority = DMA_Priority_High;//高优先级
49.     // DMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Enable; //FIFO 模式
50.     // DMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_Full;//使用全 FIFO
51.     DMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_Single;//外设突发单次传输
52.     DMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_Single;//存储器突发单次传输
53.     DMA_Init(DMA2_Stream1, &DMA_InitStructure);//初始化 DMA Stream
54.
55. }
56. //DCMI 初始化
57. void My_DCMI_Init(void)
58. {
59.     GPIO_InitTypeDef GPIO_InitStructure;
60.     NVIC_InitTypeDef NVIC_InitStructure;
61.
62.     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA|RCC_AHB1Periph_GPIOB|RCC_AHB1Periph_GPIOC|R
        CC_AHB1Periph_GPIOE, ENABLE);//使能 GPIOA B C E 时钟
63.     RCC_AHB2PeriphClockCmd(RCC_AHB2Periph_DCMI,ENABLE);//使能 DCMI 时钟
64.     //PA4/6 初始化设置
65.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4|GPIO_Pin_6;//PA4/6 复用功能输出
66.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF; //复用功能输出
67.     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;//推挽输出
68.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;//100MHz
69.     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;//上拉
70.     GPIO_Init(GPIOA, &GPIO_InitStructure);//初始化
71.
72.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_7|GPIO_Pin_6;// PB6/7 复用功能输出
73.     GPIO_Init(GPIOB, &GPIO_InitStructure);//初始化
74.
75.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6|GPIO_Pin_7|GPIO_Pin_8|GPIO_Pin_9|GPIO_Pin_11;/
        /PC6/7/8/9/11 复用功能输出
76.     GPIO_Init(GPIOC, &GPIO_InitStructure);//初始化
77.
78.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5|GPIO_Pin_6;//PE5/6 复用功能输出
79.     GPIO_Init(GPIOE, &GPIO_InitStructure);//初始化
80.
81.     GPIO_PinAFConfig(GPIOA,GPIO_PinSource4,GPIO_AF_DCMI); //PA4,AF13 DCMI_HSYNC
82.     GPIO_PinAFConfig(GPIOA,GPIO_PinSource6,GPIO_AF_DCMI); //PA6,AF13 DCMI_PCLK
83.     GPIO_PinAFConfig(GPIOB,GPIO_PinSource7,GPIO_AF_DCMI); //PB7,AF13 DCMI_VSYNC
84.     GPIO_PinAFConfig(GPIOC,GPIO_PinSource6,GPIO_AF_DCMI); //PC6,AF13 DCMI_D0
85.     GPIO_PinAFConfig(GPIOC,GPIO_PinSource7,GPIO_AF_DCMI); //PC7,AF13 DCMI_D1

```

```

86.     GPIO_PinAFConfig(GPIOC,GPIO_PinSource8,GPIO_AF_DCMI); //PC8,AF13  DCMI_D2
87.     GPIO_PinAFConfig(GPIOC,GPIO_PinSource9,GPIO_AF_DCMI); //PC9,AF13  DCMI_D3
88.     GPIO_PinAFConfig(GPIOC,GPIO_PinSource11,GPIO_AF_DCMI); //PC11,AF13  DCMI_D4
89.     GPIO_PinAFConfig(GPIOB,GPIO_PinSource6,GPIO_AF_DCMI); //PB6,AF13  DCMI_D5
90.     GPIO_PinAFConfig(GPIOE,GPIO_PinSource5,GPIO_AF_DCMI); //PE5,AF13  DCMI_D6
91.     GPIO_PinAFConfig(GPIOE,GPIO_PinSource6,GPIO_AF_DCMI); //PE6,AF13  DCMI_D7
92.
93.     DCMI_DeInit();//清除原来的设置
94.
95.     DCMI_InitStructure.DCMI_CaptureMode=DCMI_CaptureMode_Continuous;//连续模式
96.     DCMI_InitStructure.DCMI_CaptureRate=DCMI_CaptureRate_All_Frame;//全帧捕获
97.     DCMI_InitStructure.DCMI_ExtendedDataMode= DCMI_ExtendedDataMode_8b;//8 位数据格式
98.     DCMI_InitStructure.DCMI_HSPolarity= DCMI_HSPolarity_Low;//HSYNC 低电平有效
99.     DCMI_InitStructure.DCMI_PCKPolarity= DCMI_PCKPolarity_Rising;//PCLK 上升沿有效
100.    DCMI_InitStructure.DCMI_SynchroMode= DCMI_SynchroMode_Hardware;//硬件同步 HSYNC,VSYNC
101.    DCMI_InitStructure.DCMI_VSPolarity=DCMI_VSPolarity_High;//VSYNC 低电平有效
102.    DCMI_Init(&DCMI_InitStructure);
103.
104.    DCMI_ITConfig(DCMI_IT_FRAME,ENABLE);//开启帧中断
105.
106.    DCMI_Cmd(ENABLE); //DCMI 使能
107.
108.    NVIC_InitStructure.NVIC_IRQChannel = DCMI_IRQn;
109.    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority=2;//抢占优先级 2
110.    NVIC_InitStructure.NVIC_IRQChannelSubPriority =3; //子优先级 2
111.    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE; //IRQ 通道使能
112.    NVIC_Init(&NVIC_InitStructure); //根据指定的参数初始化 VIC 寄存器、
113. }
114. //DCMI,启动传输
115. void DCMI_Start(void)
116. {
117.     //对于一些型号彩屏，需要修改下窗口，要么-1，要么不减 1
118.     LCD_Set_Window(0, 0, tftlcd_data.width-1, tftlcd_data.height-1); //设置点的位置
119.     DMA_Cmd(DMA2_Stream1, ENABLE);//开启 DMA2,Stream1
120.     DCMI_CaptureCmd(ENABLE);//DCMI 捕获使能
121. }
122. //DCMI,关闭传输
123. void DCMI_Stop(void)
124. {
125.     DCMI_CaptureCmd(DISABLE);//DCMI 捕获使关闭
126.
127.     while(DCMI->CR&0X01); //等待传输结束
128.
129.     DMA_Cmd(DMA2_Stream1,DISABLE);//关闭 DMA2,Stream1

```

```

130. }
131. //////////////////////////////////////////////////
132. //以下两个函数,供 usmart 调用,用于调试代码
133.
134. //DCMI 设置显示窗口
135. //sx,sy;LCD 的起始坐标
136. //width,height:LCD 显示范围.
137. void DCMI_Set_Window(u16 sx,u16 sy,u16 width,u16 height)
138. {
139.     DCMI_Stop();
140.     LCD_Clear(WHITE);
141.     LCD_Set_Window(sx,sy,width,height);
142.     OV5640_OutSize_Set(4,0,width,height);
143.
144.     DMA_Cmd(DMA2_Stream1,ENABLE);    //开启 DMA2,Stream1
145.
146.     DCMI_CaptureCmd(ENABLE);//DCMI 捕获使能
147. }
148.
149. //通过 usmart 调试,辅助测试用.
150. //pclk/hsync/vsync:三个信号的有限电平设置
151. void DCMI_CR_Set(u8 pclk,u8 hsync,u8 vsync)
152. {
153.     DCMI_DeInit();//清除原来的设置
154.
155.     DCMI_InitStructure.DCMI_CaptureMode=DCMI_CaptureMode_Continuous;//连续模式
156.     DCMI_InitStructure.DCMI_CaptureRate=DCMI_CaptureRate_All_Frame;//全帧捕获
157.     DCMI_InitStructure.DCMI_ExtendedDataMode= DCMI_ExtendedDataMode_8b;//8 位数据格式
158.     DCMI_InitStructure.DCMI_HSPolarity= hsync<<6;//HSYNC 低电平有效
159.     DCMI_InitStructure.DCMI_PCKPolarity= pclk<<5;//PCLK 上升沿有效
160.     DCMI_InitStructure.DCMI_SynchroMode= DCMI_SynchroMode_Hardware;//硬件同步 HSYNC,VSYNC
161.     DCMI_InitStructure.DCMI_VSPolarity=vsync<<7;//VSYNC 低电平有效
162.     DCMI_Init(&DCMI_InitStructure);
163.
164.     DCMI_CaptureCmd(ENABLE);//DCMI 捕获使能
165.     DCMI_Cmd(ENABLE);    //DCMI 使能
166. }

```

其中：DCMI_IRQHandler 函数，用于处理帧中断，可以实现帧率统计（需要定时器支持）和 JPEG 数据处理等。DCMI_DMA_Init 函数，则用于配置 DCMI 的 DMA 传输，其外设地址固定为：DCMI->DR，而存储器地址可变（LCD 或者 SRAM）。DMA 被配置为循环模式，一旦开启，DMA 将不停的循环传输数据。My_DCMI_Init 函数用于初始化 STM32F4 的 DCMI 接口，这是根据前面小节提到的配置步骤进

行配置的。最后,DCMI_Start 和 DCMI_Stop 两个函数,用于开启或停止 DCMI 接口。

其他部分代码我们就不再细说了,请大家参考实验例程。

3.4 main.c

最后,打开 main.c 文件,代码如下:

```
1.  #include "system.h"
2.  #include "SysTick.h"
3.  #include "usart.h"
4.  #include "led.h"
5.  #include "key.h"
6.  #include "tftlcd.h"
7.  #include "time.h"
8.  #include "ov5640.h"
9.  #include "dcmi.h"
10.
11.
12.  u8 ovx_mode=0;                //bit0:0,RGB565 模式;1,JPEG 模式
13.
14.  #define jpeg_buf_size 31*1024    //定义 JPEG 数据缓存 jpeg_buf 的大小(*4 字节)
15.  __align(4) u32 jpeg_buf[jpeg_buf_size]; //JPEG 数据缓存 buf
16.  volatile u32 jpeg_data_len=0;    //buf 中的 JPEG 有效数据长度
17.  volatile u8 jpeg_data_ok=0;    //JPEG 数据采集完成标志
18.                                //0,数据没有采集完;
19.                                //1,数据采集完了,但是还没处理;
20.                                //2,数据已经处理完成了,可以开始下一帧接收
21.  //JPEG 尺寸支持列表
22.  const u16 jpeg_img_size_tbl[][2]=
23.  {
24.      160,120,    //QQVGA
25.      320,240,    //QVGA
26.      640,480,    //VGA
27.      800,600,    //SVGA
28.      1024,768,   //XGA
29.      1280,800,   //WXGA
30.      1440,900,   //WXGA+
31.      1280,1024,  //SXGA
32.      1600,1200,  //UXGA
33.      1920,1080,  //1080P
34.      2048,1536,  //QXGA
35.      2592,1944,  //500W
```

```

36. };
37. const u8*EFFECTS_TBL[7]={ "Normal", "Cool", "Warm", "B&W", "Yellowish ", "Inverse", "Greenish"};
    //7 种特效
38. const u8*JPEG_SIZE_TBL[12]={ "QQVGA", "QVGA", "VGA", "SVGA", "XGA", "WXGA", "WXGA+", "SXGA", "UXGA",
    "1080P", "QXGA", "500W"}; //JPEG 图片
39.
40.
41. //处理 JPEG 数据
42. //当采集完一帧 JPEG 数据后,调用此函数,切换 JPEG BUF.开始下一帧采集.
43. void jpeg_data_process(void)
44. {
45.     if(ovx_mode&0X01)    //只有在 JPEG 格式下,才需要做处理.
46.     {
47.         if(jpeg_data_ok==0) //jpeg 数据还未采集完?
48.         {
49.             DMA_Cmd(DMA2_Stream1, DISABLE); //停止当前传输
50.             while (DMA_GetCmdStatus(DMA2_Stream1) != DISABLE){} //等待 DMA2_Stream1 可配置
51.             jpeg_data_len=jpeg_buf_size-DMA_GetCurrDataCounter(DMA2_Stream1); //得到此次数据传
                输的长度
52.
53.             jpeg_data_ok=1; //标记 JPEG 数据采集完按成,等待其他函数处理
54.         }
55.         if(jpeg_data_ok==2) //上一次的 jpeg 数据已经被处理了
56.         {
57.             DMA2_Stream1->NDTR=jpeg_buf_size;
58.             DMA_SetCurrDataCounter(DMA2_Stream1, jpeg_buf_size); //传输长度为 jpeg_buf_size*4 字
                节
59.             DMA_Cmd(DMA2_Stream1, ENABLE); //重新传输
60.             jpeg_data_ok=0; //标记数据未采集
61.         }
62.     }else
63.     {
64.         LCD_Set_Window(0, 0, tftlcd_data.width-1, tftlcd_data.height-1);
65.     }
66. }
67. //JPEG 测试
68. //JPEG 数据,通过串口 1 发送给电脑.
69. void jpeg_test(void)
70. {
71.     u32 i, jpgstart, jpglen;
72.     u8 *p;
73.     u8 key, headok=0;
74.     u8 effect=0, contrast=2;
75.     u8 size=2; //默认是 QVGA 640*480 尺寸

```

```

76.     u8 msgbuf[15];           //消息缓存区
77.     LCD_Clear(WHITE);
78.     FRONT_COLOR=RED;
79.     LCD_ShowString(30,50,200,16,16,"PRECHIN-STM32");
80.     LCD_ShowString(30,70,200,16,16,"OV5640 JPEG Mode");
81.     LCD_ShowString(30,100,200,16,16,"KEY0:Contrast");           //对比度
82.     LCD_ShowString(30,120,200,16,16,"KEY1:Saturation");         //色彩饱和度
83.     LCD_ShowString(30,140,200,16,16,"KEY2:Effects");           //特效
84.     LCD_ShowString(30,160,200,16,16,"KEY_UP:Size");           //分辨率设置
85.     sprintf((char*)msgbuf,"JPEG Size:%s",JPEG_SIZE_TBL[size]);
86.     LCD_ShowString(30,180,200,16,16,msgbuf);           //显示当前 JPEG 分辨率
87.
88.     //自动对焦初始化
89.     OV5640_RGB565_Mode();           //RGB565 模式
90.     OV5640_Focus_Init();
91.
92.     OV5640_JPEG_Mode();           //JPEG 模式
93.
94.     OV5640_Light_Mode(0);           //自动模式
95.     OV5640_Color_Saturation(3); //色彩饱和度 0
96.     OV5640_Brightness(4);           //亮度 0
97.     OV5640_Contrast(3);           //对比度 0
98.     OV5640_Sharpness(33);           //自动锐度
99.     OV5640_Focus_Constant(); //启动持续对焦
100.
101.     My_DCMI_Init();           //DCMI 配置
102.     DCMI_DMA_Init((u32)&jpeg_buf,jpeg_buf_size,DMA_MemoryDataSize_Word,DMA_MemoryInc_Enable)
        ;//DCMI DMA 配置
103.     OV5640_OutSize_Set(4,0,jpeg_img_size_tbl[size][0],jpeg_img_size_tbl[size][1]); //设置输出
        尺寸
104.     DCMI_Start();           //启动传输
105.     while(1)
106.     {
107.         if(jpeg_data_ok==1) //已经采集完一帧图像了
108.         {
109.             p=(u8*)jpeg_buf;
110.             // printf("jpeg_data_len:%d\r\n",jpeg_data_len*4); //打印帧率
111.             LCD_ShowString(30,210,210,16,16,"Sending JPEG data..."); //提示正在传输数据
112.             jpglen=0;           //设置 jpg 文件大小为 0
113.             headok=0;           //清除 jpg 头标记
114.             for(i=0;i<jpeg_data_len*4;i++) //查找 0xFF,0xD8 和 0xFF,0xD9, 获取 jpg 文件大小
115.             {
116.                 if((p[i]==0xFF)&&(p[i+1]==0xD8)) //找到 FF D8
117.                 {

```



```

118.         jpgstart=i;
119.         headok=1;    //标记找到 jpg 头(FF D8)
120.     }
121.     if((p[i]==0xFF)&&(p[i+1]==0xD9)&&headok)//找到头以后,再找 FF D9
122.     {
123.         jpglen=i-jpgstart+2;
124.         break;
125.     }
126. }
127. if(jpglen) //正常的 jpeg 数据
128. {
129.     p+=jpgstart;        //偏移到 0xFF,0xD8 处
130.     for(i=0;i<jpglen;i++) //发送整个 jpg 文件
131.     {
132.         while((USART1->SR&0x40)==0); //循环发送,直到发送完毕
133.         USART1->DR=p[i];
134.         key=KEY_Scan(0);
135.         if(key)break;
136.     }
137. }
138. if(key) //有按键按下,需要处理
139. {
140.     LCD_ShowString(30,210,210,16,16,"Quit Sending data "); //提示退出数据传输
141.     switch(key)
142.     {
143.         case KEY0_PRESS: //对比度设置
144.             contrast++;
145.             if(contrast>6)contrast=0;
146.             OV5640_Contrast(contrast);
147.             sprintf((char*)msgbuf,"Contrast:%d",(signed char)contrast-3);
148.             break;
149.         case KEY1_PRESS: //执行一次自动对焦
150.             OV5640_Focus_Single();
151.             break;
152.         case KEY2_PRESS: //特效设置
153.             effect++;
154.             if(effect>6)effect=0;
155.             OV5640_Special_Effects(effect); //设置特效
156.             sprintf((char*)msgbuf,"%s",EFFECTS_TBL[effect]);
157.             break;
158.         case KEY_UP_PRESS: //JPEG 输出尺寸设置
159.             size++;
160.             if(size>11)size=0;

```

```

161.                OV5640_OutSize_Set(16,4,jpeg_img_size_tbl[size][0],jpeg_img_size_t
    1[size][1]); //设置输出尺寸
162.                sprintf((char*)msgbuf,"JPEG Size:%s",JPEG_SIZE_TBL[size]);
163.                break;
164.            }
165.            LCD_Fill(30,180,239,190+16,WHITE);
166.            LCD_ShowString(30,180,210,16,16,msgbuf); //显示提示内容
167.            delay_ms(800);
168.        }else LCD_ShowString(30,210,210,16,16,"Send data complete!!"); //提示传输结束设
    置
169.            jpeg_data_ok=2; //标记 jpeg 数据处理完了,可以让 DMA 去采集下一帧了.
170.        }
171.    }
172. }
173. //RGB565 测试
174. //RGB 数据直接显示在 LCD 上面
175. void rgb565_test(void)
176. {
177.     u8 key;
178.     u8 effect=0,contrast=2,fac;
179.     u8 scale=1;    //默认是全尺寸缩放
180.     u8 msgbuf[15]; //消息缓存区
181.     LCD_Clear(WHITE);
182.     FRONT_COLOR=RED;
183.     LCD_ShowString(30,50,200,16,16,"PRECHIN-STM32");
184.     LCD_ShowString(30,70,200,16,16,"OV5640 RGB565 Mode");
185.
186.     LCD_ShowString(30,100,200,16,16,"KEY0:Contrast");    //对比度
187.     LCD_ShowString(30,130,200,16,16,"KEY1:Saturation"); //色彩饱和度
188.     LCD_ShowString(30,150,200,16,16,"KEY2:Effects");    //特效
189.     LCD_ShowString(30,170,200,16,16,"KEY_UP:FullSize/Scale"); //1:1 尺寸(显示真实尺寸)/全尺
    寸缩放
190.
191.    //自动对焦初始化
192.    OV5640_RGB565_Mode(); //RGB565 模式
193.    OV5640_Focus_Init();
194.
195.    OV5640_Light_Mode(0); //自动模式
196.    OV5640_Color_Saturation(3); //色彩饱和度 0
197.    OV5640_Brightness(4); //亮度 0
198.    OV5640_Contrast(3);    //对比度 0
199.    OV5640_Sharpness(33); //自动锐度
200.    OV5640_Focus_Constant(); //启动持续对焦
201.

```

```

202.     My_DCMI_Init();           //DCMI 配置
203.     DCMI_DMA_Init((u32)&TFTLCD->LCD_DATA,1,DMA_MemoryDataSize_HalfWord,DMA_MemoryInc_Disabl
e); //DCMI DMA 配置
204.     OV5640_OutSize_Set(4,0,tftlcd_data.width,tftlcd_data.height);
205.     DCMI_Start();           //启动传输
206.     while(1)
207.     {
208.         key=KEY_Scan(0);
209.         if(key)
210.         {
211.             if(key!=KEY1_PRESS)DCMI_Stop(); //非 KEY1 按下,停止显示
212.             switch(key)
213.             {
214.                 case KEY0_PRESS:      //对比度设置
215.                     contrast++;
216.                     if(contrast>6)contrast=0;
217.                     OV5640_Contrast(contrast);
218.                     sprintf((char*)msgbuf,"Contrast:%d",(signed char)contrast-3);
219.                     break;
220.                 case KEY1_PRESS:      //执行一次自动对焦
221.                     OV5640_Focus_Single();
222.                     break;
223.                 case KEY2_PRESS:      //特效设置
224.                     effect++;
225.                     if(effect>6)effect=0;
226.                     OV5640_Special_Effects(effect); //设置特效
227.                     sprintf((char*)msgbuf,"%s",EFFECTS_TBL[effect]);
228.                     break;
229.                 case KEY_UP_PRESS:    //1:1 尺寸(显示真实尺寸)/缩放
230.                     scale=!scale;
231.                     if(scale==0)
232.                     {
233.                         fac=800/tftlcd_data.height; //得到比例因子
234.                         OV5640_OutSize_Set((1280-fac*tftlcd_data.width)/2,(800-fac*tftlcd_d
ata.height)/2,tftlcd_data.width,tftlcd_data.height);
235.                         // fac=800/tftlcd_data.width; //得到比例因子
236.                         // OV5640_OutSize_Set((800-fac*tftlcd_data.width)/2,(1280-fac*tftlcd_d
ata.height)/2,tftlcd_data.width,tftlcd_data.height);
237.                         sprintf((char*)msgbuf,"Full Size 1:1");
238.                     }else
239.                     {
240.                         OV5640_OutSize_Set(4,0,tftlcd_data.width,tftlcd_data.height);
241.                         sprintf((char*)msgbuf,"Scale");
242.                     }

```

```

243.             break;
244.         }
245.         if(key!=KEY1_PRESS) //非 KEY1 按下
246.         {
247.             LCD_ShowString(30,50,210,16,16,msgbuf);//显示提示内容
248.             delay_ms(800);
249.             DCMI_Start(); //重新开始传输
250.         }
251.     }
252.     delay_ms(10);
253. }
254. }
255.
256. int main(void)
257. {
258.     u8 key;
259.     u8 t;
260.
261.     SysTick_Init(168); //初始化延时函数
262.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);//设置系统中断优先级分组 2
263.     USART1_Init(921600); //初始化串口波特率为 921600
264.     LED_Init(); //初始化 LED
265.     TFTLCD_Init(); //LCD 初始化
266.     KEY_Init(); //按键初始化
267.     TIM3_Init(10000-1,8400-1);//10Khz 计数,1 秒钟中断一次
268.
269.     FRONT_COLOR=RED;//设置字体为红色
270.     LCD_ShowString(30,50,200,16,16,"PRECHIN-STM32");
271.     LCD_ShowString(30,70,200,16,16,"OV5640 TEST");
272.     LCD_ShowString(30,90,200,16,16,"www.prechin.net");
273.     while(OV5640_Init())//初始化 OV2640
274.     {
275.         printf("OV5640 ERR\r\n");
276.         LCD_ShowString(30,130,240,16,16,"OV5640 ERR");
277.         delay_ms(200);
278.         LCD_Fill(30,130,239,170,WHITE);
279.         delay_ms(200);
280.     }
281.     printf("OV5640 OK\r\n");
282.     printf("KEY0:RGB565 KEY1:JPEG\r\n");
283.     LCD_ShowString(30,130,200,16,16,"OV5640 OK");
284.     while(1)
285.     {
286.         key=KEY_Scan(0);

```

```

287.         if(key==KEY0_PRESS)           //RGB565 模式
288.         {
289.             ovx_mode=0;
290.             break;
291.         }else if(key==KEY1_PRESS)       //JPEG 模式
292.         {
293.             ovx_mode=1;
294.             break;
295.         }
296.         t++;
297.         if(t==100)
298.         {
299.             LED1=!LED1;
300.             LCD_ShowString(30,150,230,16,16,"KEY0:RGB565  KEY1:JPEG"); //闪烁显示提示信息
301.         }
302.         if(t==200)
303.         {
304.             LCD_Fill(30,150,210,150+16,WHITE);
305.             t=0;
306.         }
307.         delay_ms(5);
308.     }
309.     if(ovx_mode)jpeg_test();
310.     else rgb565_test();
311. }

```

这里我们定义了一个数组 jpeg_buf (124KB)，用来存储 JPEG 数据，因为 STM32F407ZGT 有 192KB RAM，你可以根据你的代码定义为其他尺寸的 buf，另外我们的 T200&T300 开发板是有外扩 1MB SRAM 的你也可以利用起来。在 main.c 里面，总共有 4 个函数：jpeg_data_process、jpeg_test、rgb565_test 和 main 函数。其中，jpeg_data_process 函数用于处理 JPEG 数据的接收，在 DCMI_IRQHandler 函数里面被调用，通过一个 jpeg_data_ok 变量与 jpeg_test 函数共同控制 JPEG 数据传送。jpeg_test 函数将 OV5640 设置为 JPEG 模式，并开启持续对焦，该函数接收 OV5640 的 JPEG 数据，并通过串口 1 发送给上位机。rgb565_test 函数将 OV5640 设置为 RGB565 模式，并将接收到的数据，直接传送给 LCD，处理过程完全由硬件实现，CPU 完全不用理会。最后，main 函数就相对简单了，大家看代码即可。

4 实验现象

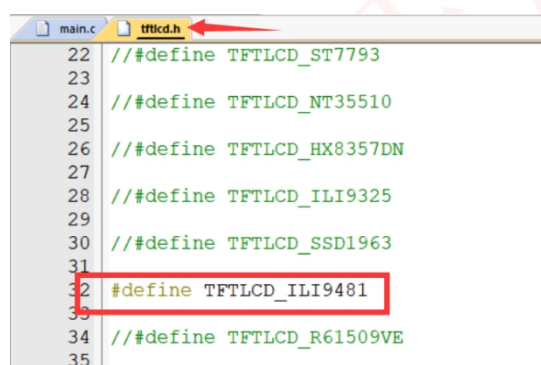
首先，请先确保硬件都已经连接好：

①PZ-OV5640 模块与天马&麒麟开发板连接（可参考硬件设计小节）

②开发板插上 TFTLCD 液晶，对于没有 TFTLCD 的用户，可通过串口摄像头工具显示图像。

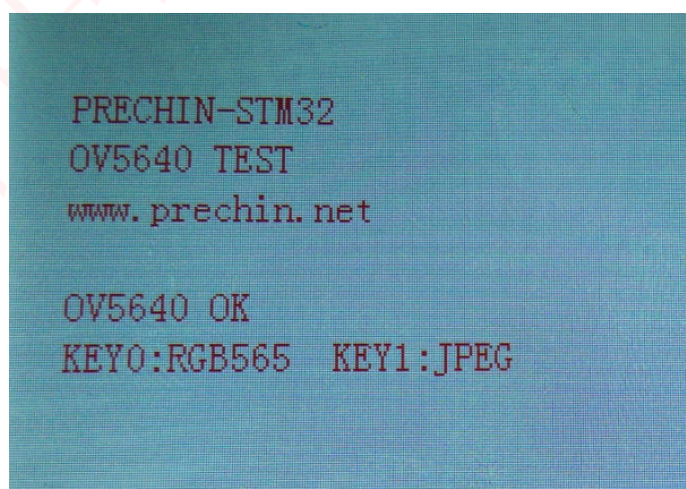
③使用 USB 线给开发板供电。

注意：如果使用普中 TFTLCD 触摸屏，用户需根据手上彩屏右上角或背面驱动型号来设置程序中使能的 TFTLCD 驱动。在 tftlcd.h 头文件中开始处进行设置，比如我使用的 TFTLCD 是 ILI9481，那么就要把这个宏定义放开，其他的注释掉。如下所示：



```
22  //#define TFTLCD_ST7793
23
24  //#define TFTLCD_NT35510
25
26  //#define TFTLCD_HX8357DN
27
28  //#define TFTLCD_ILI9325
29
30  //#define TFTLCD_SSD1963
31
32  #define TFTLCD_ILI9481
33
34  //#define TFTLCD_R61509VE
35
```

代码编译成功之后，我们将代码下载到 STM32 开发板上。（注意：以下的功能测试图片以麒麟开发板为展示）LCD 界面显示如下图所示：



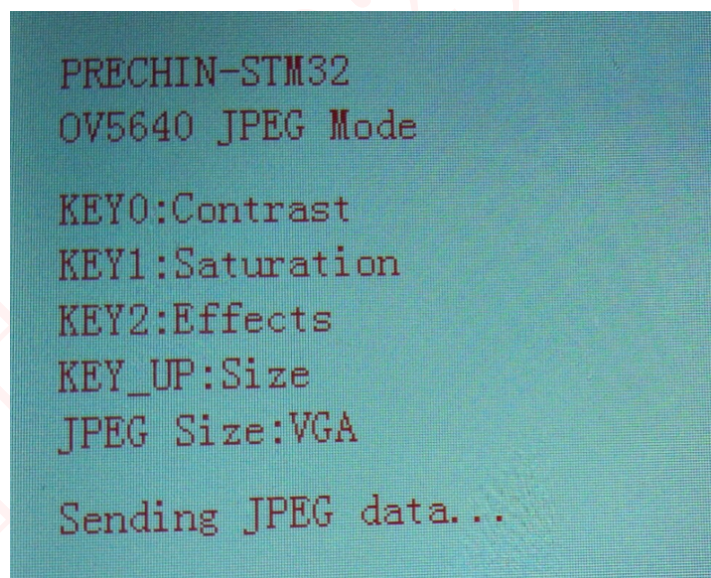
在 OV5640 初始化成功后，屏幕提示选择模式，此时我们可以按 KEY0，进入 RGB565 模式测试，也可以按 KEY1，进入 JPEG 模式测试。

当按 KEY0 后，选择 RGB565 模式，LCD 满屏显示压缩放后的图像，如下图所示：



此时，可以按下 KEY_UP 切换为 1:1 显示。同时还可以通过 KEY0 按键，设置对比度；KEY1 按键，执行一次自动对焦；KEY2 按键，设置特效。

当按 KEY1 后，选择 JPEG 模式，此时屏幕显示 JPEG 数据传输进程，如下图所示：



默认条件下，图像分辨率是 VGA(640*480)的，此时通过串口 1 发送图像数据到上位机。我们打开上位机软件“\3--软件工具\串口&网络摄像头软件\CAM.exe”，选择正确的串口，即开发板上 CH340 串口，然后波特率设置为 921600，打开即可收到 STM32 传过来的图片了，如下图所示：



我们可以通过 KEY_UP 设置输出图像的尺寸（QQVGA~VGA）。通过 KEY0 按键，设置对比度；KEY1 按键，执行一次自动对焦；KEY2 按键，设置特效。

在 time.c 的 TIM3 中断函数中，我们把帧率输出注释掉了，如果要查看多少帧，可以自行打开。

5 其他

（1）购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

（2）资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

（3）技术支持

普中官网：www.prechin.cn

普中论坛：www.prechin.net

技术电话：0755-21509063（转技术）